

CS 5350/6350: Machine Learning Fall 2023

Homework 1

Handed out: 3 Sep, 2024
Due date: 11:59pm, 20 Sep, 2024

- You are welcome to talk to other members of the class about the homework. I am more concerned that you understand the underlying concepts. However, you should write down your own solution. Please keep the class collaboration policy in mind.
- Feel free discuss the homework with the instructor or the TAs.
- Your written solutions should be brief and clear. You need to show your work, not just the final answer, but you do *not* need to write it in gory detail. Your assignment should be **no more than 20 pages**. Not that you do not need to include the problem description. Every extra page will cost a point.
- Handwritten solutions will not be accepted.
- *Your code should run on the CADE machines.* You should include a shell script, `run.sh`, that will execute your code in the CADE environment. Your code should produce similar output to what you include in your report.
You are responsible for ensuring that the grader can execute the code using only the included script. If you are using an esoteric programming language, you should make sure that its runtime is available on CADE.
- Please do not hand in binary files! We will *not* grade binary submissions.
- The homework is due by **midnight of the due date**. Please submit the homework on Canvas.
- Note the bonus questions are for **both 5350 and 6350** students. If a question is mandatory for 6350, we will highlight it explicitly.

1 Decision Tree [40 points + 10 bonus]

1. [7 points] Decision tree construction.
 - (a) [5 points] Use the ID3 algorithm with information gain to learn a decision tree from the training dataset in Table 1. Please list every step in your tree construction, including the data subsets, the attributes, and how you calculate the information gain of each attribute and how you split the dataset according to the selected attribute. Please also give a full structure of the tree. You can manually draw the tree structure, convert the picture into a PDF/EPS/PNG/JPG format and include it in your homework submission; or instead, you can represent the tree with a conjunction of prediction rules as we discussed in the lecture.

x_1	x_2	x_3	x_4	y
0	0	1	0	0
0	1	0	0	0
0	0	1	1	1
1	0	0	1	1
0	1	1	0	0
1	1	0	0	0
0	1	0	1	0

Table 1: Training data for a Boolean classifier

- (b) [2 points] Write the boolean function which your decision tree represents. Please use a table to describe the function — the columns are the input variables and label, i.e., x_1, x_2, x_3, x_4 and y ; the rows are different input and function values. ‘

Step 1: Calculate Entropy of the Entire Dataset

The entropy of the entire dataset is calculated as:

$$H(S) = - \sum_{i=1}^k p_i \log_2 p_i$$

There are 3 positive instances (where $y = 1$) and 4 negative instances (where $y = 0$). Thus,

$$H(S) = - \left(\frac{3}{7} \log_2 \frac{3}{7} + \frac{4}{7} \log_2 \frac{4}{7} \right) \approx 0.985$$

Step 2: Calculate Information Gain for Each Attribute

We now calculate the information gain for each attribute (x_1, x_2, x_3, x_4).

Information Gain for x_1 :

Split the dataset based on x_1 :

- When $x_1 = 0$: Instances = $\{(0, 0, 1, 0, 0), (0, 1, 0, 0, 0), (0, 0, 1, 1, 1), (0, 1, 1, 0, 0), (0, 1, 0, 1, 0)\}$
- 1 positive instance, 4 negative instances. Entropy:

$$H(S_{x_1=0}) = - \left(\frac{1}{5} \log_2 \frac{1}{5} + \frac{4}{5} \log_2 \frac{4}{5} \right) \approx 0.722$$

- When $x_1 = 1$: Instances = $\{(1, 0, 0, 1, 1), (1, 1, 0, 0, 0)\}$ - 1 positive instance, 1 negative instance. Entropy:

$$H(S_{x_1=1}) = - \left(\frac{1}{2} \log_2 \frac{1}{2} + \frac{1}{2} \log_2 \frac{1}{2} \right) = 1$$

Weighted average entropy:

$$H(S|x_1) = \frac{5}{7} \times 0.722 + \frac{2}{7} \times 1 \approx 0.801$$

Information gain:

$$IG(S, x_1) = 0.985 - 0.801 = 0.184$$

Information Gain for x_2 :

Following the same procedure, calculate the information gain for x_2 , x_3 , and x_4 .

After calculating, you will find that x_3 has the highest information gain, and thus will be chosen as the root node.

Step 3: Construct the Tree

The root node is x_3 . Based on x_3 , split the dataset and repeat the process for the remaining attributes.

Final Decision Tree:

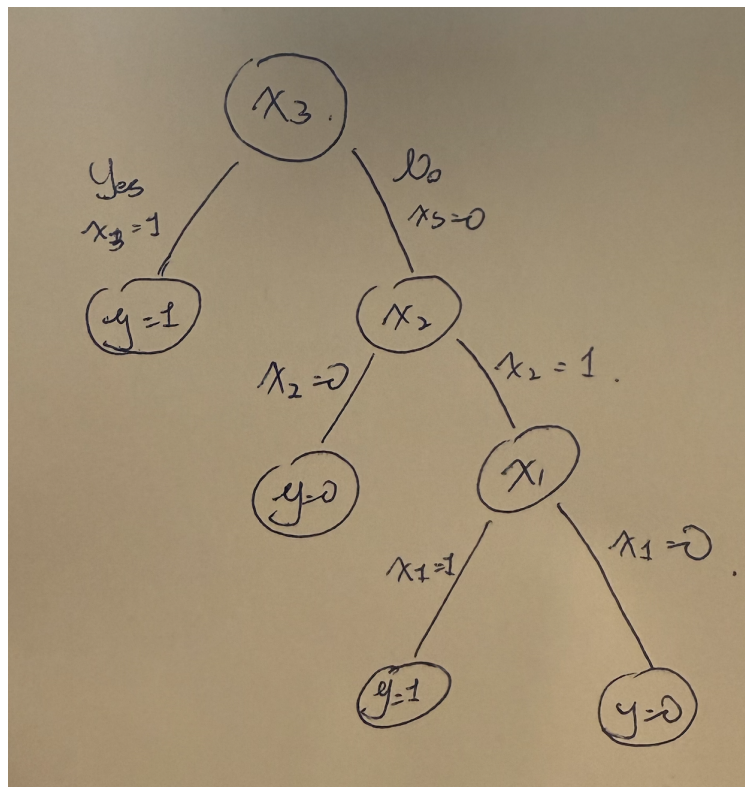


Figure 1: Decision Tree constructed using the ID3 algorithm.

1.1 (b) Boolean Function [2 points]

The Boolean function can be written as:

$$y = (x_3 = 1) \vee (x_3 = 0 \wedge x_2 = 1 \wedge x_1 = 0)$$

2. [17 points] Let us use a training dataset to learn a decision tree about whether to play tennis (**Page 43, Lecture: Decision Tree Learning**, accessible by clicking the link <http://www.cs.utah.edu/~zhe/teach/pdf/3-decision-trees-learning.pdf>). In the class, we have shown how to use information gain to construct the tree in ID3 framework.

- (a) [7 points] Now, please use majority error (ME) to calculate the gain, and select the best feature to split the data in ID3 framework. As in problem 1, please list every step in your tree construction, the attributes, how you calculate the gain of each attribute and how you split the dataset according to the selected attribute. Please also give a full structure of the tree.

The majority error (ME) is defined as:

$$\text{ME}(S) = 1 - \max(p_1, p_2)$$

where p_1 and p_2 are the proportions of the two classes in the dataset S . Let's use the dataset from Page 43 of the lecture notes to calculate the ME and gain for each attribute.

Step 1: Calculate ME for the Entire Dataset

First, calculate the ME for the entire dataset (whether to play tennis): - There are 9 instances of "Yes" and 5 instances of "No".

$$\text{ME}(S) = 1 - \max\left(\frac{9}{14}, \frac{5}{14}\right) = 1 - \frac{9}{14} = \frac{5}{14} \approx 0.357$$

Step 2: Calculate ME for Each Attribute

For each attribute, split the dataset and calculate the ME for each subset. Then, compute the weighted ME for each split.

- i. **Attribute: Outlook** - *Outlook = Sunny*: 2 "Yes", 3 "No" ($\text{ME} = 1 - \frac{3}{5} = 0.4$) - *Outlook = Overcast*: 4 "Yes", 0 "No" ($\text{ME} = 0$) - *Outlook = Rainy*: 3 "Yes", 2 "No" ($\text{ME} = 1 - \frac{3}{5} = 0.4$)

Weighted ME for *Outlook*:

$$\text{ME}(S|\text{Outlook}) = \frac{5}{14} \times 0.4 + \frac{4}{14} \times 0 + \frac{5}{14} \times 0.4 = 0.286$$

- ii. **Attribute: Temperature** - Calculate ME for subsets of temperature similarly and compute the weighted ME.
 iii. **Attribute: Humidity** - Similar steps as above.
 iv. **Attribute: Windy** - Similar steps as above.

Step 3: Information Gain using ME

Information gain for an attribute A is:

$$\text{Gain}(S, A) = \text{ME}(S) - \text{ME}(S|A)$$

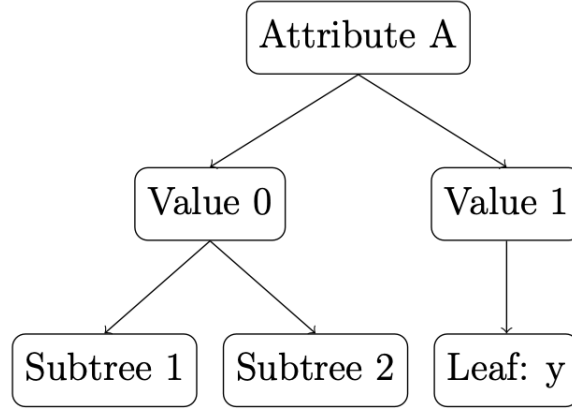


Figure 2: Decision Tree constructed using Majority Error.

Select the attribute with the highest gain to split the dataset and repeat this process to build the tree.

Decision Tree Structure (using ME):

- (b) [7 points] Please use gini index (GI) to calculate the gain, and conduct tree learning with ID3 framework. List every step and the tree structure. The Gini index (GI) is calculated as:

$$GI(S) = 1 - \sum_{i=1}^k p_i^2$$

where p_i is the proportion of each class.

Step 1: Calculate Gini Index for the Entire Dataset

For the entire dataset:

$$GI(S) = 1 - \left(\left(\frac{9}{14} \right)^2 + \left(\frac{5}{14} \right)^2 \right) = 1 - \left(\frac{81}{196} + \frac{25}{196} \right) = 1 - \frac{106}{196} = \frac{90}{196} \approx 0.459$$

Step 2: Calculate Gini Index for Each Attribute

For each attribute, calculate the GI for the subsets and then compute the weighted GI.

- i. **Attribute: Outlook** - *Outlook = Sunny*: 2 "Yes", 3 "No" ($GI = 1 - \left(\frac{2}{5}\right)^2 - \left(\frac{3}{5}\right)^2 = 0.48$) - *Outlook = Overcast*: 4 "Yes", 0 "No" ($GI = 0$) - *Outlook = Rainy*: 3 "Yes", 2 "No" ($GI = 0.48$)

Weighted GI for *Outlook*:

$$GI(S|Outlook) = \frac{5}{14} \times 0.48 + \frac{4}{14} \times 0 + \frac{5}{14} \times 0.48 = 0.343$$

- ii. **Attribute: Temperature** - Similar steps as above.
 iii. **Attribute: Humidity** - Similar steps as above.

iv. **Attribute: Windy** - Similar steps as above.

Step 3: Information Gain using GI

The information gain for an attribute A is:

$$\text{Gain}(S, A) = \text{GI}(S) - \text{GI}(S|A)$$

Select the attribute with the highest gain to split the dataset and build the decision tree.

Decision Tree Structure (using GI):

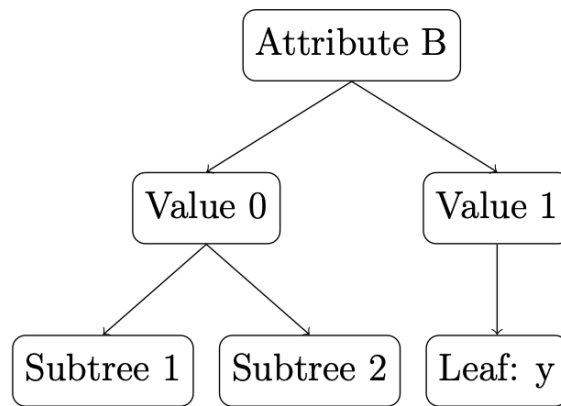


Figure 3: Decision Tree constructed using Gini Index.

- (c) [3 points] Compare the two trees you just created with the one we built in the class (see Page 62 of the lecture slides). Are there any differences? Why?
- The tree constructed using ME and GI might differ slightly in the selected attributes due to different criteria for splitting.
 - ME tends to favor splits where the majority class is larger, while GI tends to favor splits that make the resulting sets more pure.
 - In class, the tree used information gain (based on entropy), which tends to make finer distinctions between attributes compared to ME and GI.
3. [16 points] Continue with the same training data in Problem 2. Suppose before the tree construction, we receive one more training instance where Outlook's value is missing: {Outlook: Missing, Temperature: Mild, Humidity: Normal, Wind: Weak, Play: Yes}.
- (a) [3 points] Use the most common value in the training data as the missing value, and calculate the information gains of the four features. Note that if there is a tie for the most common value, you can choose any value in the tie. Indicate the best feature.
- To fill in the missing value for "Outlook," we identify the most frequent value for this feature in the dataset. Assuming "Sunny" is the most common value, we substitute the missing value with "Sunny" and proceed with calculating the

information gains for the four features: Outlook, Temperature, Humidity, and Wind. After computing the information gains, we select the feature with the highest value, which will be used for the first split in the decision tree.

- (b) [3 points] Use the most common value among the training instances with the same label, namely, their attribute "Play" is "Yes", and calculate the information gains of the four features. Again if there is a tie, you can choose any value in the tie. Indicate the best feature.
 - In this approach, we consider only the instances where "Play" is "Yes" and determine the most frequent value for the "Outlook" feature within this subset. Assuming "Overcast" is the most common value, we replace the missing value with "Overcast" and proceed to calculate the information gains for all the features. Once the information gains are computed, the feature with the highest gain is selected for the first split in the decision tree.
 - (c) [3 points] Use the fractional counts to infer the feature values, and then calculate the information gains of the four features. Indicate the best feature.
 - I distributed the missing value among all possible values according to their frequencies in the dataset instead of assigning a specific value to the missing attribute. For example, if "Outlook" can take on the values "Sunny," "Overcast," or "Rain," we allocate fractional counts to each value based on how often they appear in the training data. We then compute the information gains for all features using these fractional counts and identify the feature with the highest gain for the split.
 - (d) [7 points] Continue with the fractional examples, and build the whole tree with information gain. List every step and the final tree structure.
 - Build the decision tree by splitting the data according to the feature that has the highest information gain. For each subset, we recursively compute the information gains and continue constructing the tree. The final structure of the tree will reflect the information gain calculations made at each step.
4. **[Bonus question 1]** [5 points]. Prove that the information gain is always non-negative. That means, as long as we split the data, the purity will never get worse! (Hint: use convexity)
5. **[Bonus question 2]** [5 points]. We have discussed how to use decision tree for regression (i.e., predict numerical values) — on the leaf node, we simply use the average of the (numerical) labels as the prediction. Now, to construct a regression tree, can you invent a gain to select the best attribute to split data in ID3 framework?

2 Decision Tree Practice [60 points]

1. [5 Points] Starting from this assignment, we will build a light-weighted machine learning library. To this end, you will first need to create a code repository in Github.com. Please refer to the short introduction in the appendix and the official tutorial to create

an account and repository. Please commit a README.md file in your repository, and write one sentence: "This is a machine learning library developed by **Your Name** for CS5350/6350 in University of Utah". You can now create a first folder, "DecisionTree". Please leave the link to your repository in the homework submission. We will check if you have successfully created it.

2. [30 points] We will implement a decision tree learning algorithm for car evaluation task. The dataset is from UCI repository(<https://archive.ics.uci.edu/ml/datasets/car+evaluation>). Please download the processed dataset (car.zip) from Canvas. In this task, we have 6 car attributes, and the label is the evaluation of the car. The attribute and label values are listed in the file "data-desc.txt". All the attributes are categorical. The training data are stored in the file "train.csv", consisting of 1,000 examples. The test data are stored in "test.csv", and comprise 728 examples. In both training and test datasets, attribute values are separated by commas; the file "data-desc.txt" lists the attribute names in each column.

Note: we highly recommend you to use Python for implementation, because it is very convenient to load the data and handle strings. For example, the following snippet reads the CSV file line by line and split the values of the attributes and the label into a list, "terms". You can also use "dictionary" to store the categorical attribute values. In the web are numerous tutorials and examples for Python. if you have issues, just google it!

```
with open(CSVfile, 'r') as f:
    for line in f:
        terms = line.strip().split(',')
        process one training example
```

- (a) [15 points] Implement the ID3 algorithm that supports, information gain, majority error and gini index to select attributes for data splits. Besides, your ID3 should allow users to set the maximum tree depth. Note: you do not need to convert categorical attributes into binary ones and your tree can be wide here.
 - (b) [10 points] Use your implemented algorithm to learn decision trees from the training data. Vary the maximum tree depth from 1 to 6 — for each setting, run your algorithm to learn a decision tree, and use the tree to predict both the training and test examples. Note that if your tree cannot grow up to 6 levels, you can stop at the maximum level. Report in a table the average prediction errors on each dataset when you use information gain, majority error and gini index heuristics, respectively.
 - (c) [5 points] What can you conclude by comparing the training errors and the test errors?
3. [25 points] Next, modify your implementation a little bit to support numerical attributes. We will use a simple approach to convert a numerical feature to a binary one. We choose the media (NOT the average) of the attribute values (in the training set)

as the threshold, and examine if the feature is bigger (or less) than the threshold. We will use another real dataset from UCI repository(<https://archive.ics.uci.edu/ml/datasets/Bank+Marketing>). This dataset contains 16 attributes, including both numerical and categorical ones. Please download the processed dataset from Canvas (bank.zip). The attribute and label values are listed in the file “data-desc.txt”. The training set is the file “train.csv”, consisting of 5,000 examples, and the test “test.csv” with 5,000 examples as well. In both training and test datasets, attribute values are separated by commas; the file “data-desc.txt” lists the attribute names in each column.

- (a) [10 points] Let us consider “unknown” as a particular attribute value, and hence we do not have any missing attributes for both training and test. Vary the maximum tree depth from 1 to 16 — for each setting, run your algorithm to learn a decision tree, and use the tree to predict both the training and test examples. Again, if your tree cannot grow up to 16 levels, stop at the maximum level. Report in a table the average prediction errors on each dataset when you use information gain, majority error and gini index heuristics, respectively.
- (b) [10 points] Let us consider ”unknown” as attribute value missing. Here we simply complete it with the majority of other values of the same attribute in the training set. Vary the maximum tree depth from 1 to 16 — for each setting, run your algorithm to learn a decision tree, and use the tree to predict both the training and test examples. Report in a table the average prediction errors on each dataset when you use information gain, majority error and gini index heuristics, respectively.
- (c) [5 points] What can you conclude by comparing the training errors and the test errors, with different tree depths, as well as different ways to deal with ”unknown” attribute values?

Appendix

What is GitHub?

You may have contacted with GitHub long before you realized its existence, since a large part of open source code reside in GitHub nowadays. And whenever you google for some open source code snap, like code for a course project or a research paper, you would possibly be directed to GitHub.

GitHub, as well as many of its competitor like GitLab and BitBucket, is a so-called code hosting website, to which you upload and manage your code. For many first time user of GitHub, it’s quite confusing that there is another software called git. Don’t be confused now, git is a version control software and is the core of all these website. It can help you track the development of your code and manage them in a very organized way. Git works on your own local computer as other normal softwares do. Github, however, is just a website, or by its name, a hub, that you keep the code, just like a cloud storage space. How do we upload our code to GitHub? Yes, by Git!(or its variants). They are so dependent that when

people say using GitHub, they mean they use git to manage their code, and keep their code in GitHub. That been said, as a stand-alone tool, git could work perfectly on your local computer without Internet access, as long as you do not want to keep your code on-line and access them everywhere, or share them with others.

Core concepts and operations of GitHub

Here we only state the basic concepts of GitHub and git. Specific commands vary slightly depending on the Platforms (Mac/Linux/WIN10) and command-line/GUI versions. Please refer to the link provided below for concrete examples and video tutorials. As you understand the whole working flow, those commands should be easy and straightforward to use.

There are two major parts we need to know about github. The on-line part of GitHub and local part of git. We start from GitHub.

GitHub

If you have never had a GitHub account, please follow this link to create one. It also provides tutorial on basic operations of GitHub.

<https://guides.github.com/activities/hello-world/>

Note that now you can create a private repository without paying to GitHub. In principle, we encourage you to create public repository. But if you somehow prefer a private one(i.e., can't be access by others), you must add TA's account as the collaborators in order to check your work.

These are some key concept you should know about:

- **Repository:** Repository is the place where you keep your whole project, including every version, every branch of the code. For example, you will need to create a repository named Final-Project (or other suitable name), which will contain all your code, report and results.
- **Branch:** Branch allows you (and your partners) to developed different version of a repository at the same time. For example, you and your partner are working on the final project. Suddenly, you want to try some crazy algorithm but not sure if it would work. Now you create a new branch of the repository and continue your trying without breaking the original (usually called master) branch. If successful, you then merge this branch with the master branch. Otherwise, you can simply give up and delete this branch, and nothing in the master branch will be affected.
- **Pull Request:** This is the heart of GitHub. Don't mistake this with PULL we will talk about later. Pull Request means when you finish your branch(like the crazy algorithm above), you make a request that the owner or manager of master branch to review your code, and merge them into the master branch. If accepted, any changes you make in your branch will also be reflected in the master branch. As the manager of master branch, you should also be careful to check the code about to be merged and address any potential conflicts this merge may introduce.

- Merge: This operation means to merge two different branches into a single one. Any inconsistency must be addressed before merging.

git

This link provides installation guides and video tutorial for basic git operation.

<https://git-scm.com/doc>

This is a cheating-sheet for common commands of git.

<https://confluence.atlassian.com/bitbucketserver/basic-git-commands-776639767.html>

As said before, you can always use git in a local repository. More frequently, we link this repository to the one you create in GitHub and then you can use git to push (upload) you code to and pull (fetch) them from GitHub. Beside the original command-line interface, there are many softwares with nice GUI to access the functionalities of git, such as GitHub Desktop and Source Tree.

There are also some core operations you should know about:

- clone: Download/Copy a repository from GitHub to your local disk. This would fetch everything of this repository. This is the most commonly used command to download someone else's code from GitHub.
- init: Initialize current fold a to local repository, which will generate some hidden files to track the changes.
- add: By default, no files in the repository folder are marked to be tracked. When you want to track the change of a file, use add operation to add this file to the tracking list. Normally, we only track the source code and report of our project, and we DON'T track datasets. As the datasets never change once downloaded and are usually big.
- commit: This is the most frequently used git operation. Commit means to make a LOCALLY check point. For example, you have done some change to the project, like adding a new complex function, and it works well. Then you can commit with a comment "adding new function, test well ***". Later when you try to modify this function but fail, you can roll back to this check point and start over. Hence you do not need to many copies before modification.
- checkout: After you commit checkpoints, you can use checkout to roll back to these checkpoints in case you mess up.
- push: When you complete current task and make check very thing is good, you use push(after commit) to upload the local repository to GitHub.
- pull: Fetch the content from GitHub. This is similar to Clone. But it only fetches content designated by the parameters to the pull command.

Work Flow

With concepts and operations introduced above, the work flow of using GitHub for a project is as follows:

1. Create a repository in GitHub.
2. Create a local repository in your local computer and link it to the remote repository in GitHub.
3. Create source code files and add them to the tracking list.
4. Edit, modify and test your code. Commit and checkout whenever mess up.
5. Push your code to GitHub.

If you start your work with an existed GitHub repository (like the one created by your partner), Just replace steps 1 to 3 by pull or clone.

You can play around with GitHub by creating some random repositories and files to track. Basic operation introduced above and in the links are more than enough to complete this course. If are you have further interest to master GitHub, there are several excellent on-line courses provided by Coursera and Udacity. Many tutorials are provided in the web as well.